.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 1
Algoritmos Greedy

17 de septiembre de 2023

Geronimo Palavecino
109681

Macarena Vita Sanchez
109377

Mariana Galdo Martinez
105658

1. Introducción

En el siguiente informe abordaremos un problema intrigante y desafiante que enfrenta el técnico de la selección campeona del mundo, Scaloni. El mismo se encuentra ante un desafío crucial: analizar los videos de sus próximos rivales de manera detallada y eficiente. Como un perfeccionista en su trabajo, Scaloni comprende la importancia de conocer cada detalle sobre los equipos adversarios, desde sus tácticas de juego hasta las necesidades de sus jugadores estrella, como Lionel Messi.

Para llevar a cabo esta tarea, Scaloni no está solo; cuenta con un grupo de ayudantes dispuestos a colaborar. Sin embargo, la clave está en la distribución inteligente del trabajo. Cada análisis de video requiere tiempo, tanto por parte de Scaloni como de sus ayudantes, y es crucial determinar el orden en el que deben abordarse los videos para minimizar el tiempo total necesario para completar todos los análisis.

Para ello creamos un algoritmo Greedy que garantiza la solución óptima a este problema, obteniendo el tiempo mínimo que puede tardar el equipo en analizar a todos sus rivales.

Resolución

2. Análisis del problema

El problema que enfrentamos se centra en la optimización de la secuencia en la que Scaloni y sus ayudantes deben ver los videos de sus próximos rivales. Tenemos cuatro elementos clave que influyen en este problema:

- Número de Rivales (n): Representa la cantidad de rivales que la selección enfrentará en los próximos partidos.
- Número de ayudantes de Scaloni (n): Representa la cantidad de ayudantes de Scaloni.
- Tiempo de Análisis de Scaloni (s_i): Este valor denota el tiempo que le lleva a Scaloni analizar en detalle un video de uno de los rivales. Es importante destacar que Scaloni siempre ve el video primero antes de que un ayudante lo analice.
- Tiempo de Análisis de los Ayudantes (a_i): Aquí se define el tiempo que le lleva a un ayudante analizar el mismo video después de que Scaloni lo haya revisado. Este tiempo es independiente del ayudante que mire el video.

Por otro lado sabemos que un ayudante no puede ver el vídeo de un rival si ese video no fue previamente visto por Scaloni. Una representación gráfica del mismo es:

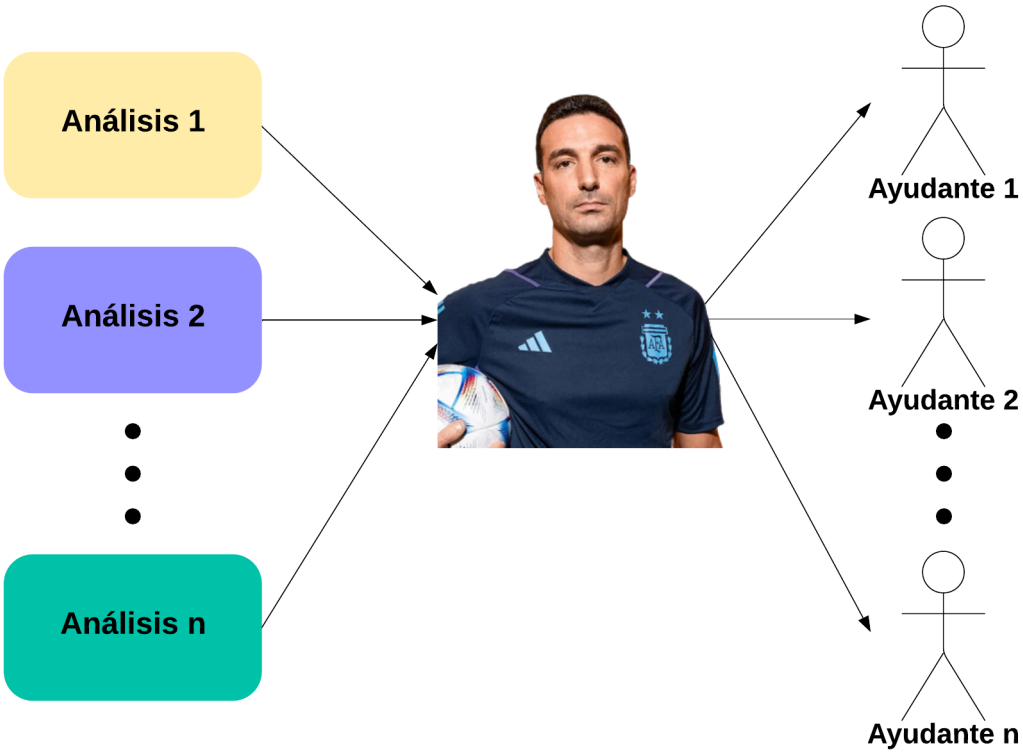


Figura 1: Contexto del problema

3. Análisis de posibles soluciones

A continuación planteamos las posibles soluciones que se nos ocurrieron antes de llegar a la solución final.

3.1. Propuesta 1

En primer lugar, consideramos el caso en que ordenamos los tiempos que tarda Scaloni en ver los videos de mayor a menor y le asignamos las tareas a Scaloni y a sus ayudantes en ese orden. Una vez ordenados, le asignamos a Scaloni cada uno de esos análisis y, cada vez que termina el i-ésimo análisis, el ayudante i empieza a realizar el análisis de ese video. Una vez que todos los ayudantes hayan terminado de ver sus videos asignados, veremos el ayudante que terminó último el video. Siguiendo este procedimiento, el tiempo en que termina ese ayudante sería el tiempo mínimo necesario para realizar todos los análisis.

Veamos un caso particular para ver cómo funcionaría. Sean los siguientes valores los valores de tiempo ordenados que dedica Scaloni y los ayudantes a analizar 3 rivales:

s_i	a_i
96	150
78	130
26	64

Graficamos el ejemplo y notamos que, para este caso en específico, se obtiene la solución optima.

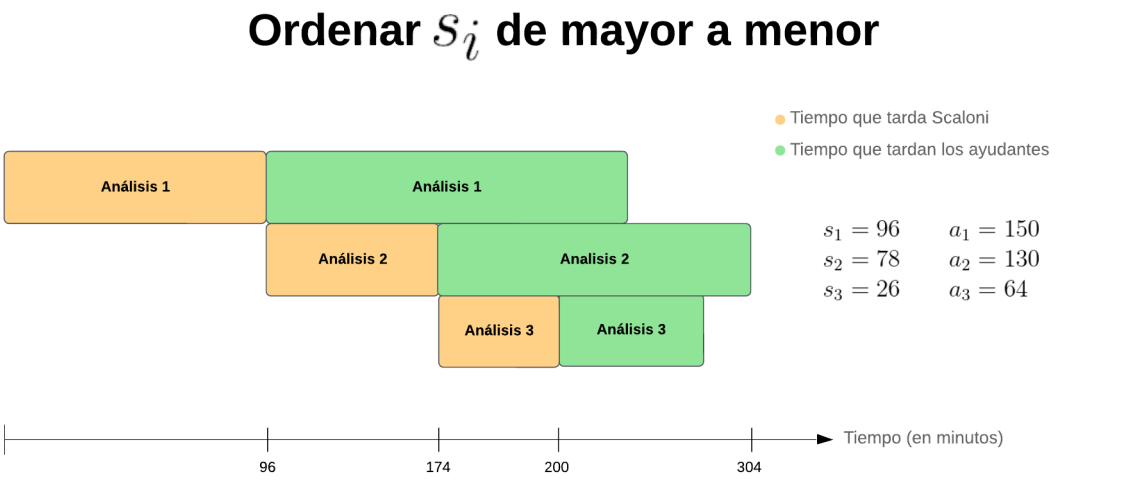


Figura 2: Propuesta 1

Rápidamente encontramos dos detalles:

- El tiempo que tarda Scaloni en ver todos los videos es independiente del orden en que los mira. Es decir, el tiempo que tarde Scaloni en terminar el análisis nunca va a variar, sin importar el orden que se elija.
- Si Scaloni tarda poco tiempo viendo un video no implica que su ayudante también lo haga. Si se da ese caso, estaríamos tardando mucho más tiempo dado que el tiempo total sería el tiempo total de Scaloni más el tiempo del ayudante que podría ser significativo y no dar con la solución óptima.

En el ejemplo anterior, el video que le llevaba más tiempo a Scaloni era también el video que más tiempo le llevaba al ayudante. Ejemplifiquemos una situación donde esto no ocurra.
En el caso de que los valores sean los siguientes,

s_i	a_i
96	120
78	150
26	64

el primer análisis no es el que requiere más tiempo tanto para Scaloni como para el ayudante, sino que el segundo análisis es el cual un ayudante va a dedicarle más tiempo.

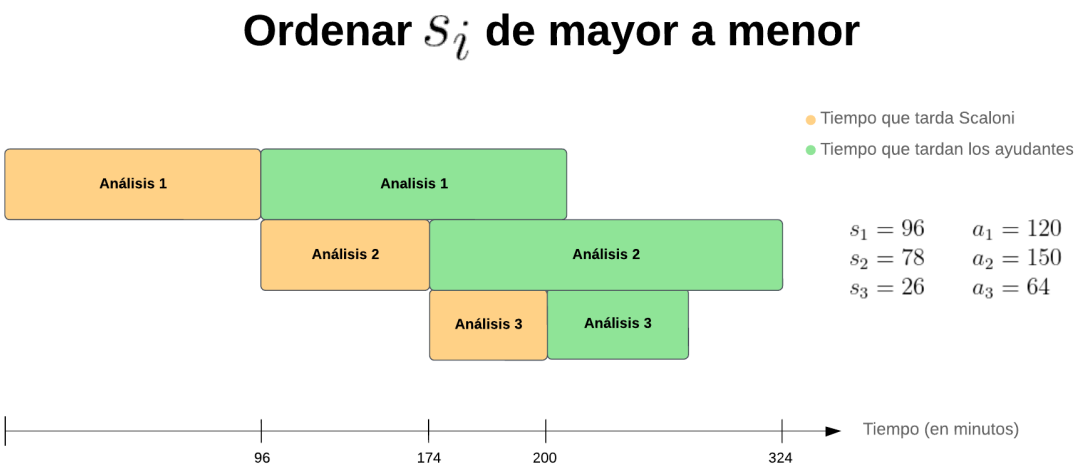


Figura 3: Contraejemplo para la propuesta 1

Aquí podemos ver que, la solución obtenida de valor 324 no es la óptima. Si se realizaba el análisis 2 primero, se hubiera obtenido una solución mejor de valor 294.

Esta situación nos lleva a plantear otra forma de encontrar la solución óptima.

3.2. Propuesta 2

En segundo lugar, consideramos ordenar el tiempo que tardan los ayudantes de mayor a menor. De esta manera, si un ayudante tarda mucho tiempo en analizar un rival, se lo hará comenzar antes que los otros. De esta forma, tardará lo menos posible. En esta propuesta como en la anterior, buscamos visualizar como se desarrolla este algoritmo con el problema.

Si vemos el contraejemplo de la propuesta anterior, los videos quedarían ordenados de la siguiente manera:

s_i	a_i
78	150
96	120
26	64

Al ordenar de esta forma, y fijarnos cuál ayudante tarda más tiempo, encontramos la solución óptima de valor 294.

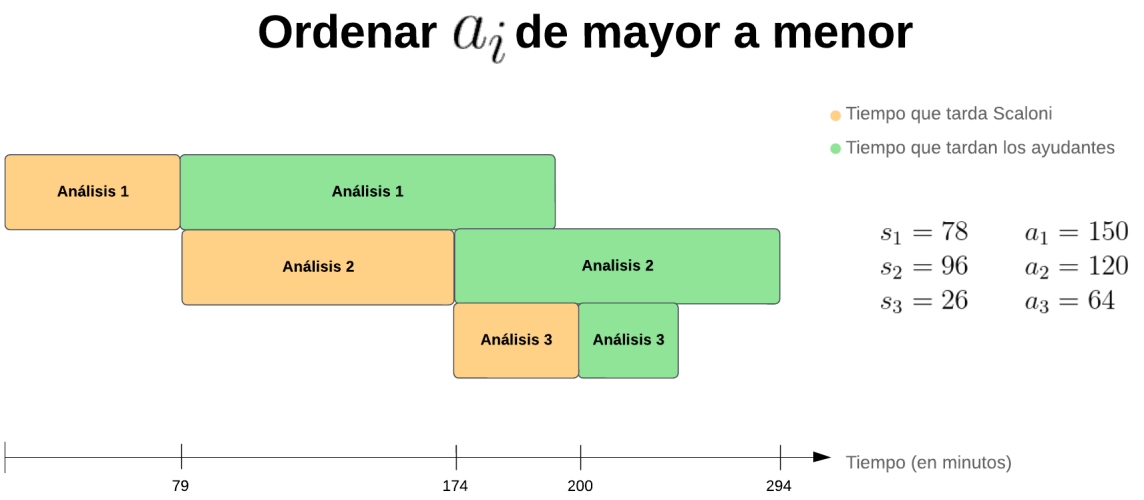


Figura 4: Ejemplo para la propuesta 2

En esta situación se hace evidente que, para los análisis más largos, los ayudantes necesitan empezar lo antes posible. Por lo tanto, Scaloni debería empezar con los análisis que requieren más tiempo de análisis por parte de los ayudantes.

4. Análisis de la propuesta

Con el objetivo de verificar que este algoritmo obtiene siempre el orden óptimo, realizamos análisis gráficos del problema:

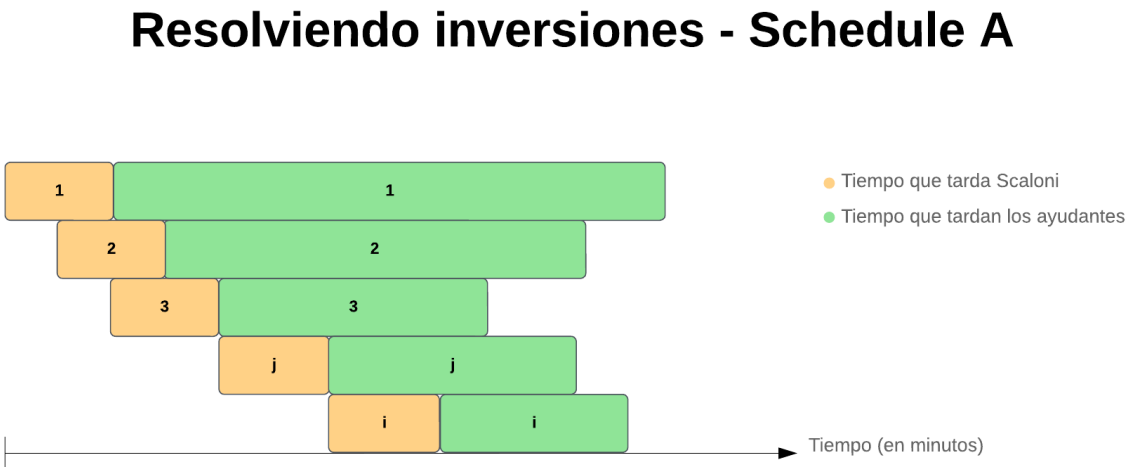


Figura 5: Orden óptimo de visualización

Este ordenamiento que seguimos para asignar las tareas al dt y a los asistentes nos da una propuesta de solución, que parecería ser la óptima. Llamaremos a esta solución posible como schedule A. Para demostrar que esta solución es la óptima, realizaremos inversiones sobre una

solución óptima hipotética, y veremos que son equivalentes.

Consideremos un schedule M que tiene un order distinto a nuestro schedule A, y ambos ordenan una cantidad n de videos a analizar. Este nuevo schedule S tiene dos videos v_i y v_j, tal que v_j se analiza luego de v_i , pero el tiempo que un asistente analiza v_i es menor que el tiempo para analizar v_j. Es decir, $a_i < a_j$. Si llamamos M' a un nuevo schedule, donde solo se intercambian el orden de los videos v_i y v_j, los tiempos en que terminan los demás asistentes se mantendrán igual, solo los asistentes i y j se verán afectados por este cambio. Como el tiempo que un ayudante tarda en analizar el video v_i es menor al tiempo para analizar v_j, en el nuevo schedule, el ayudante i termina antes que el ayudante j en el schedule original M.

Por lo tanto, si hay un caso como el anterior, donde i es menor a j, pero $a_i < a_j$, diremos que el schedule tiene una inversión. Al realizar intercambios como el detallado anteriormente, la cantidad de inversiones decrece, sin afectar al tiempo que se tarda en ver todos los videos. Por lo tanto, podremos convertir M en M' que será equivalente a A. De esta manera, podemos decir que el tiempo en que se finaliza de ver todos los videos en A no es mayor que el tiempo para cualquier otro schedule M. De esta manera podemos confirmar que la solución obtenida con el schedule A es óptima.

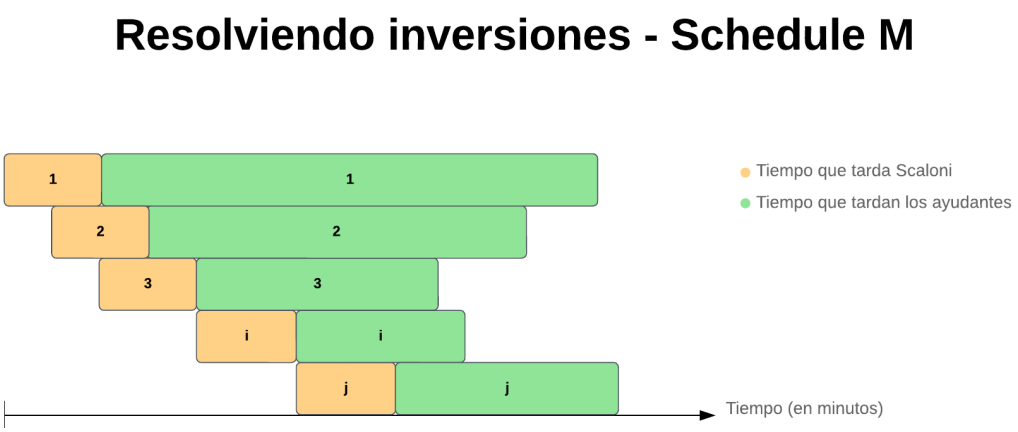


Figura 6: Schedule M

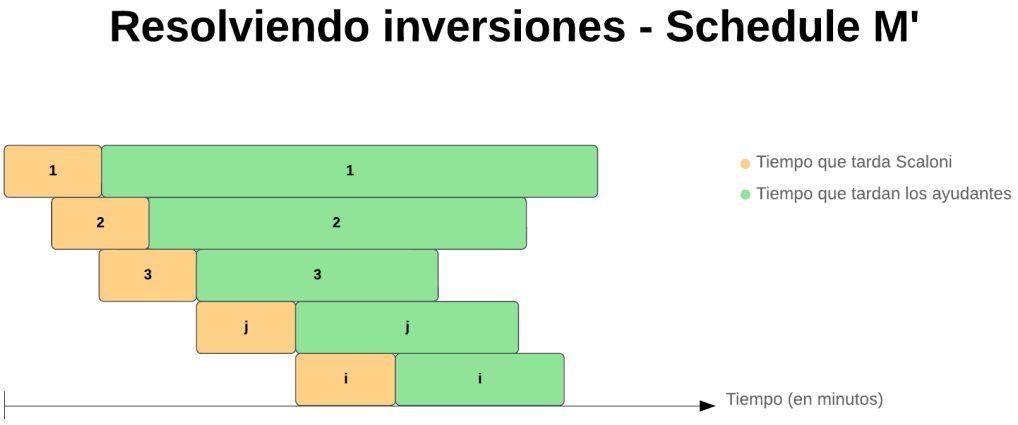


Figura 7: Schedule M' después del intercambio

5. Algoritmo para obtener el óptimo

A continuación presentamos el algoritmo Greedy implementado para su posterior análisis de complejidad. Al momento de crear el algoritmo dividimos nuestro problema en dos partes: el ordenamiento de los tiempos según los ayudantes y el posterior relevamiento de tiempos para obtener el tiempo mínimo.

5.1. Algoritmo de ordenamiento - MergeSort

A continuación se presenta el algoritmo de MergeSort implementado:

```
def merge_sort(tiempos):
    if len(tiempos) <= 1:
        return tiempos
    izq = tiempos[0:len(tiempos) // 2]
    der = tiempos[len(tiempos) // 2:]
    merge_izq = merge_sort(izq)
    merge_der = merge_sort(der)

    sorted = sort(merge_izq, merge_der)
    return sorted

def merge(izq, der):
    resultado = []
    i = 0
    j = 0
    while i < len(izq) and j < len(der):
        if izq[i][1] > der[j][1]:
            resultado.append(izq[i])
            i += 1
            continue
        resultado.append(der[j])
        j += 1
    while i < len(izq):
        resultado.append(izq[i])
        i = i + 1
    while j < len(der):
        resultado.append(der[j])
        j = j + 1
    return resultado
```

Para obtener la complejidad del algoritmo utilizamos el Teorema Maestro: Donde:

- A: Cantidad de llamados recursivos, en este caso A=2
- B: Proporción del tamaño original con el que llamamos recursivamente, en este caso B=2
- $O(n^c)$: Costo de partir y juntar, en este caso $c = 1$.

La ecuación de recurrencia final es :

$$\mathcal{T}(n) = 2\mathcal{T}\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

La complejidad final es :

$$O(n * log(n)) \tag{1}$$

5.2. Algoritmo de tiempo mínimo Greedy

```
def tiempo_minimo(tiempos):                          #O(n * log(n) + n)
  tiempos = merge_sort(tiempos)                      #O(n * log(n))
  tiempo_scaloni = 0                                 #O(1)
  tiempo_maximo = 0                                  #O(1)
  for tiempo in tiempos:                             #O(n)
    tiempo_scaloni += tiempo[0]                      #O(1)
    tiempo_actual = tiempo_scaloni + tiempo[1]       #O(1)
    if tiempo_actual > tiempo_maximo:                #O(1)
      tiempo_maximo = tiempo_actual                  #O(1)
  return tiempo_maximo                               #O(1)
```

Como se puede observar este algoritmo contiene la complejidad de Mergesort más la de recorrer el arreglo de tiempos, nos queda entonces:

$$T(n) = O(n * log(n)) + O(n) = O(n * log(n)) \tag{2}$$

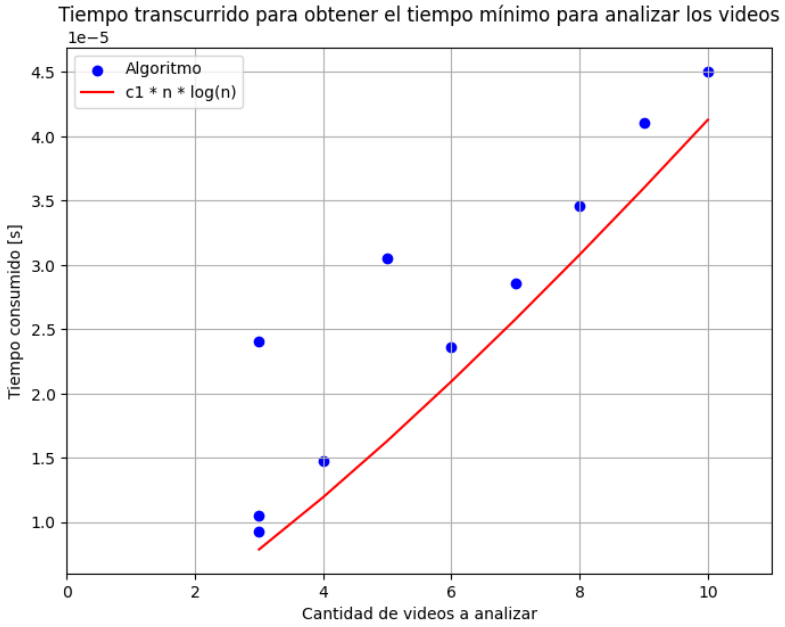
Pues nos quedamos con el peor de los casos.

5.3. Análisis de optimalidad y tiempos

A continuación se realizaron una serie de pruebas para verificar que nuestro algoritmo obtiene los resultados óptimos para distintos sets de datos. Una vez que el algoritmo funcionaba correctamente, graficamos los tiempos de ejecución del algoritmo en función a la cantidad de datos.

Realizando un análisis de la variabilidad de los valores s_i y a_i notamos que ésta no afecta el tiempo en el que nuestro algoritmo encuentra la solución ni la optimalidad del mismo. Esto se debe a que realiza un ordenamiento de mayor a menor de los datos para luego iterar linealmente, buscando un máximo acumulativo dentro de los mismos. No obstante, el volumen de datos a analizar, como veremos en el análisis de complejidad temporal, aumenta los tiempos de ejecución del algoritmo, debido a que se aumenta la cantidad de operaciones a realizar.

Por otro lado podemos observar que se respeta la complejidad propuesta en el problema siendo lineal logarítmica, patrón que se repetirá al hacer el mismo análisis con un volumen de datos notablemente mayor.

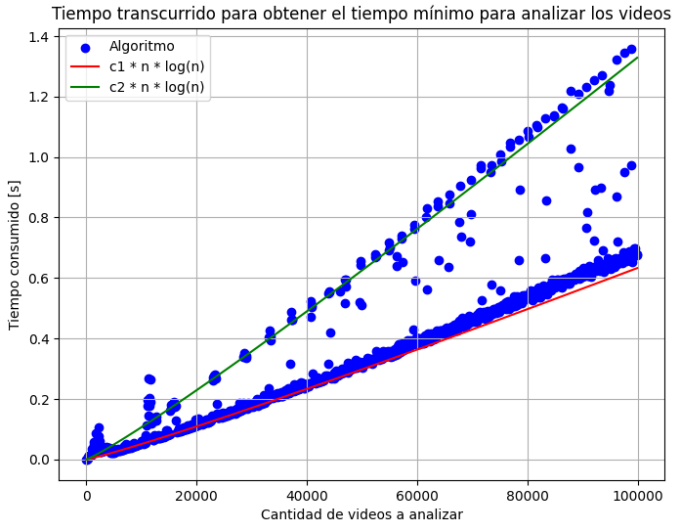


6. Mediciones

Se realizaron mediciones en base a arreglos de diferentes largos, yendo de 100 en 100 elementos, donde los elementos en cada caso fueron generados por los valores pseudoaleatorios del lenguaje (el módulo `random`).

Para comprobar que la complejidad de nuestro algoritmo se encuentra correctamente calculada, realizamos mediciones del tiempo que el algoritmo desarrollado tardaba en encontrar la solución óptima para diferentes cantidades de elementos en un arreglo.

Para realizar estas mediciones, se realizó un bucle, en el cual se generaba un arreglo de n elementos aleatorios (Utilizando el paquete `random` de python), para el cual se buscaba el tiempo mínimo para ver todos los videos. Para esta búsqueda, se calculaba el tiempo que el algoritmo tardaba en llegar a la solución, y se lo guardaba en un arreglo. Este proceso se repitió para todas las iteraciones del bucle, incrementando la cantidad de elementos por arreglo, comenzando por 0 elementos, y aumentando hasta un máximo de 100000 elementos. Luego, una vez que se poseía el arreglo con todos los tiempos, para los 1000 arreglos diferentes, se procedió a graficar los tiempos tardados, contra la cantidad de elementos en el arreglo. Sobre esto, se superpuso la gráfica de dos funciones $f(n) = c_i * n * log(n)$ [1], para poder ver la similitud en tendencia del algoritmo, con la expresión de la complejidad calculada.



En el gráfico se pueden observar dos grandes grupos de puntos. Por un lado, se observa un grupo, con notable mayor densidad, que sigue la tendencia de la curva roja. Esto nos indica que los tiempos obtenidos por nuestro algoritmo poseen un crecimiento lineal logarítmico. Por lo tanto, nos asegura que la complejidad de nuestro algoritmo sea la calculada.

Por otro lado, se observa un segundo grupo, de menor densidad, que sigue la tendencia de la curva verde. Estos puntos poseen valores de tiempo calculados de casi el doble respecto del otro grupo. Sin embargo, también siguen una tendencia lineal logarítmica (en este caso, con una pendiente poco mayor a dos veces la de la curva roja). Este segundo grupo de tiempos se puede explicar como consecuencia de estar midiendo tiempos de operación dentro de una computadora, la cual posee múltiples procesos al mismo tiempo, para los cuales puede dedicar más o menos recursos (o llegar a congelarlos completamente), dependiendo de la importancia que posean para el sistema en un momento dado.

[1] La constante c_i se utiliza para poder ajustar el conjunto de llegada de la función, para que sea en magnitud, similar a los valores obtenidos al calcular los tiempos del algoritmo. A un nivel físico, se la puede tomar como el producto del valor que representa el tiempo que tarda (en segundos) cada una de las operaciones de la computadora, por la constante de conversión del logaritmo natural (logaritmo utilizado por numpy) a logaritmo base 2 (logaritmo que se desarrolla al realizar la recursividad en mergesort). Este valor fue obtenido de forma experimental.

7. Conclusiones

En primer lugar, bajo ciertas condiciones, un enfoque de algoritmo Greedy demuestra ser eficiente y efectivo para encontrar soluciones óptimas locales de manera rápida y sucesiva, para así llegar a una solución global del problema. Esto lleva a la creación de un algoritmo simple y fácil de entender que puede abordar el problema de manera efectiva.

En nuestro caso, hemos podido identificar y analizar el problema en cuestión, comparándolo con una variación del problema de programación de tareas y reducción de latencia (scheduling). Este enfoque nos permitió encontrar contraejemplos que validan la funcionalidad y la corrección de nuestra solución.

Finalmente, logramos comparar la complejidad temporal calculada con mediciones de tiempo de nuestro algoritmo, logrando observar una buena aproximación a la analítica.